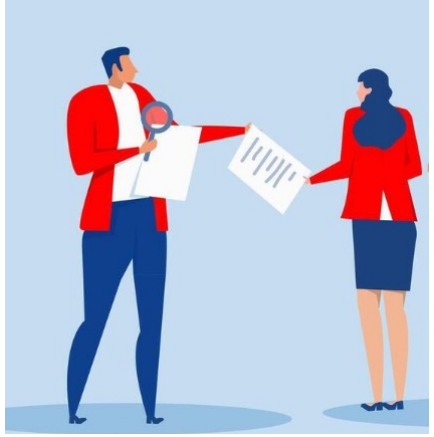
Function introduction in C++

function

- So far, we put all codes in main function. But as problem becomes more complicate, the size of codes become bigger.
- Analog: in a small company, every task is run by CEO. But as business grows, it is a good idea to divide the duties and hire additional employees.
- Let one employee finish a specific task.
- The CEO then coordinate among corresponding employees to finish the job.



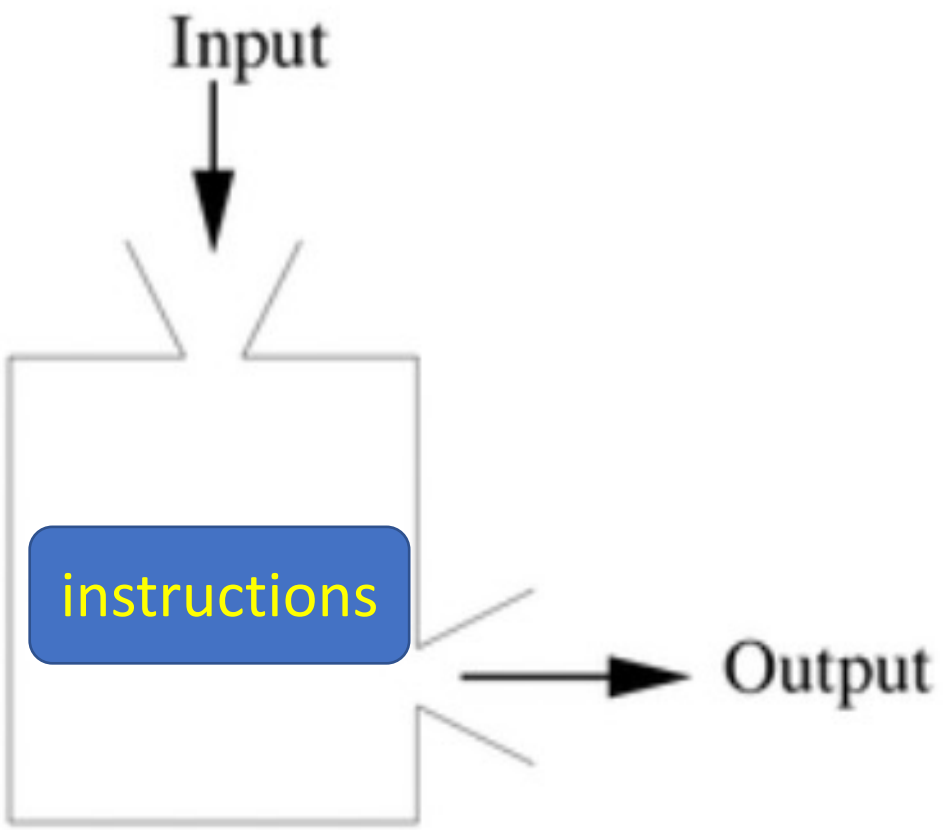
function: II

- A function is like an employee, performing a specified task. Specify duty for an employee is like to define a function, that is, what the inputs and expected return of a function, and how to process the inputs to get the return.
- Function main is also an employee, but its duty now changes to call the corresponding functions to finish the job. Previously main function just call system provided functions/methods. Now we need to write user-defined functions, and call these functions.

A function is a named block of code that performs one specific task

- Takes **input** (optional)
- work on **instructions**
- Produces **output** (optional)

(this slide is from ChatGPT)



Input and return

- Device concentrates on one useful task.
  - For example, a bread machine.
- Input is like the materials needed to feed into the device for it to start working.
- Return is like the results given to the user that uses the device.

Input and Return: quarter changer

Input dollar bill and return quarters

one input,
one return

Input and Return: vending machine

Input: money in bills and coins

Return: a metro card with that amount of money

two inputs, one return

Input and Return: alarm clock

What is input?

What is the return?

one input,
no return

We use alarm time as input, get some audio effect (alarm sound), but no physical return.

What is a function

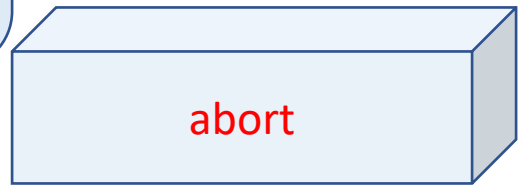
A function like a recipe, it is a set of instructions that process the given input, and return something if necessary.

Components of function

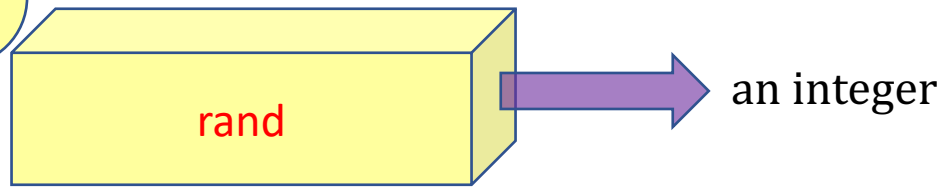
- A function has a meaningful name, normally as a verb (print, toString, …).
- zero, one, or many inputs
- zero or one return
  - If we have more than one return, pack them in a composite type -- a structure composed of several types. So, the return is still one, but it contains many stuffs.
- A function has some functionality.
  - One function normally concentrates only on one task.

Existing functions with no input

Input: none
Return: none
Functionality: Aborts the current process, producing an abnormal program termination.

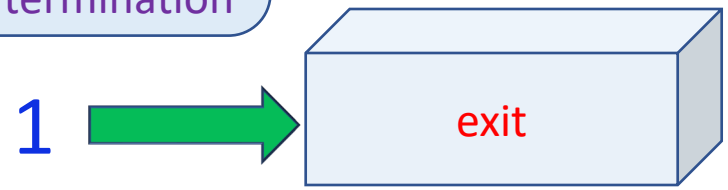


Input: none
Return: an integer
Functionality: Returns a pseudo-random integral number in the range between 0 and RAND_MAX.

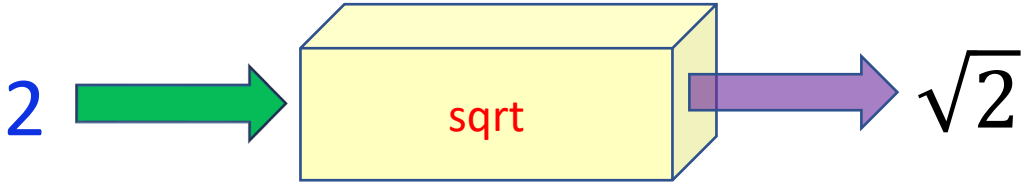


Existing functions with one input

Input: a number
Return: none
Functionality: Causes normal program termination

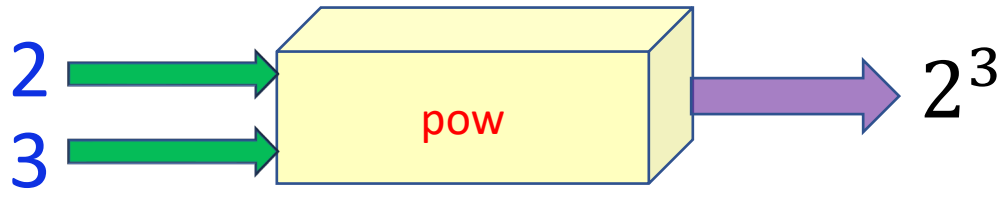


Input: a number
Return: the square root
Functionality: calculate the square root of input

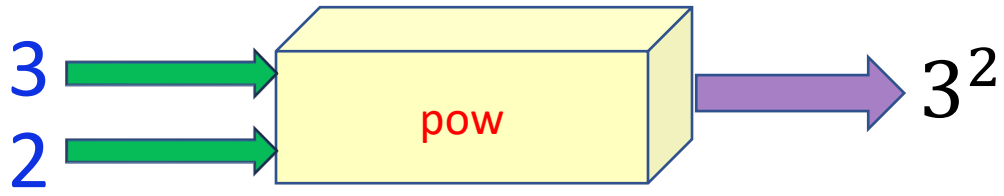


Existing functions with two inputs

pow(2, 3) order matters



pow(3, 2)



Have we defined any function before?

```
int main() {
    // TODO Auto-generated method stub
    // Your code goes here ...
}
```

- What is the name of the function?
- What is the input of the function?
- Does this function return anything?
- Where to define the functionality of the function?

Define a function

```
int main() {
    // Your code goes here …
}
```

A function has a header, followed by a body.

1. Method header

    1.1 return **type**; **if there is no return**, use void

    1.2 function name (should be meaningful)

    1.3 Declare input parameters inside a pair of parentheses. **If there is no input**, use a pair of empty parentheses.

2. Function body contains code to implement specific functionality, enclosed in a pair of braces {}.

{} is needed even the method body has only one statement.

Why function?

- Abstraction
- Simplify the code
  - isPrime(int n)

Why function: II

- Modular design: subdivides a system into smaller parts (modules) that can be independently created and used.
- debug (find which method has errors) and test independently
- Expand
- Define once; use many times.
- Define functions for other programmers to use; use functions defined by other programmers.

Rules for designing function

- It needs experience. Each function has an easily defined singular purpose.
- When reading code from a single function, you should be able to remember what it does from beginning to end.
- Rule of thumb: a method normally contains only 25-30 lines.

Caller vs. callee

- Callee: the function being called
- Caller: the function that calls another function
- The roles of caller and callee are relative.
  - main function calls isPrime function, and isPrime method can call isFactor function.
  - So, isPrime is a caller for isFactor and a callee for main

When a function is called

- Caller saves
  - local variables
  - return point (what is the next instruction to run after finishing calling a method)
- Caller passes parameter list to callee.
- Caller yields control to callee.
- Callee works with the input parameters and returns something if required.
- Callee yields control to caller.
- Caller continue to work from the return point.

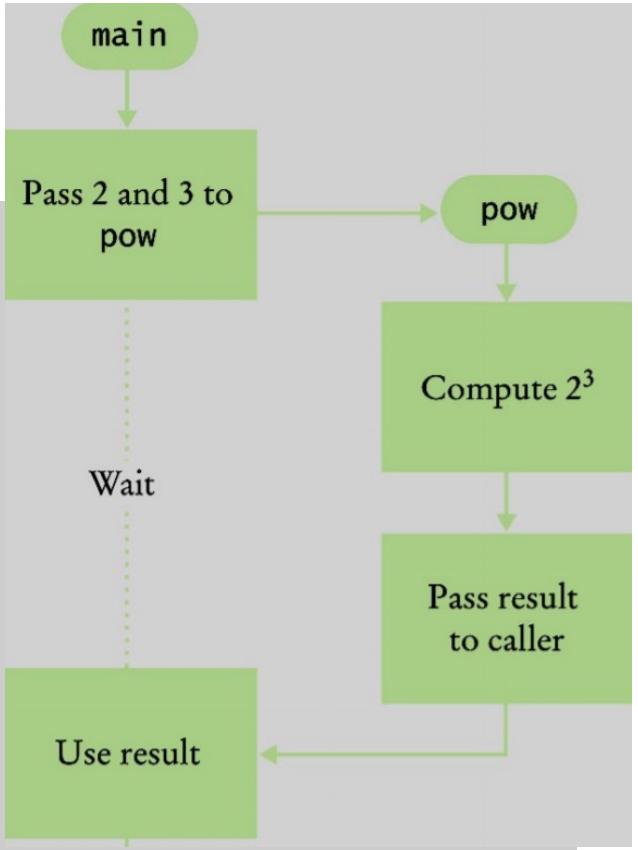
Relation between caller and callee

- Given the following code.

```
#include<iostream>
#include<cmath> //pow
using namespace std;

int main()
{
    double result = pow(2, 3);

    cout << "pow(2, 3) = " << result << endl;
    return 0;
}
```



Related: function in math

- Definition: $f(x) = x^2$, where f is function name, x is parameter, and x^2 is the functionality of a function (aka, what a function can do).
- Call the above function to find out f(3), it will return 3^2, which is 9. Do NOT call function as f(~~int~~ 3).
- In math, only work with numbers, but in computer, the types can range from int, double, char, …, to string, and so on.

Function in C++

- In C++, to define a function,

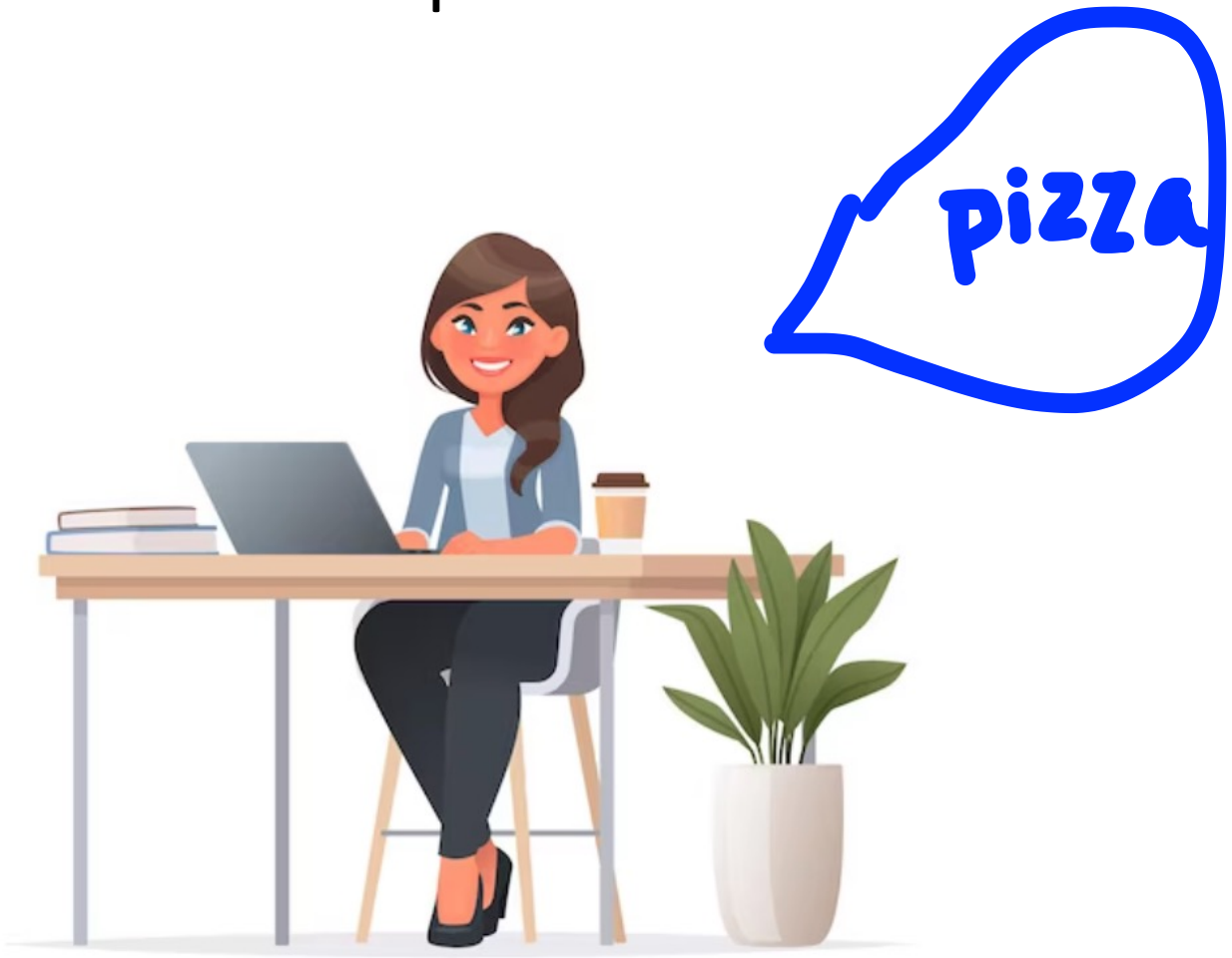
```
return_type function_name(parameter_list)
    //there can be 0, 1, or more parameters,
    //parameter list is type parameter_name separated by comma (,)
{
    //functionality, what to do with the parameters
}
```

- In C++, to call a function

```
functionName(actual_parameter_list); //if return type is not void, can put the return of function to a variable with the same type as return type.
```

Call function example

Call function example



Call function example

Call function example

Code: https://onlinegdb.com/fuaVg5xna

```
#include <iostream>

//function prototype
int add(int a, int b);

int main() {
    int m = 1;
    int n = 2;
    int total = add(m, n);
    std::cout << "total = " << total << std::endl;
    return 0;
}
```

Code: https://onlinegdb.com/fuaVg5xna

```
//function implementation
int add(int a, int b) {
    return a + b;
}
```

Exercise

- Write a function to find out whether an integer is prime or not. Then call this function in main method to print out all the prime integers from 1 to 100.
- By definition, a prime is a positive integer that has only 1 and itself as factors.
  - So, 1 is not a prime by definition.

Exercise

- Given an integer n, define 1 and n as trivial factors of n. The factors that are not trivial are called non-trivial factors.
  - So, a prime has no non-trivial factors.
  - TODO: find out whether an integer has non-trivial factors or not.

Relationship between non-trivial factor and prime

- Non-trivial factor candidates of integer n are _____.
- How to find out whether a candidate is ACTUALLY a factor of n?
- What if we find the first non-trivial factor of n?
  - What are non-trivial factor candidates of 25?
  - What is the first non-trivial factor of 25?
- What if none of the non-trivial factor candidates is a factor of n?
  - What are non-trivial factor candidates of 11?

Existence of non-trivial factors or not

- If n has at least a non-trivial factor, then n is _____.
- If n has NO non-trivial factor, then n is _____.

Given a batch of eggs,

- If there is at least a bad egg, then this batch of eggs does not pass quality check.
- If there is no bad egg, then this batch of eggs pass quality check.

One and Done

- In isPrime function
- Once number n has a non-trivial factor, n cannot be a prime, return false IMMEDIATELY.
- Only when every non-trivial factor candidate is tested, and n does not have any of those candidate as actual factor, return true.
- In checking eggs
- Once a bad egg is found, the batch of eggs are declared “NOT good”
- Only when EVERY egg is good, the batch of eggs can be declared “good”.

Existence of non-trivial factor or not: II

- At least one non-trivial factor

Prime? NO

- None non-trivial factor

Prime? Yes

- Prime number does not allow existence of non-trivial factors.

factor? factor? factor? …

NO NO YES

factor? factor? factor? … factor?

NO NO NO NO

- At least one bad egg

- No bad egg

- Quality control does not allow existence of bad eggs.

Which of the following pseudo-code is correct?

```
bool QAPassed(eggs a) {
    for each egg in a {
        if the current egg is bad {
            return false;
        }
        else return true;
    }
}
```

```
bool QAPassed(eggs a) {
    for each egg in a {
        if the current egg is bad {
            return false;
        }
    }

    return true;
}
```

candidates for non-trivial smaller factors of n

- Suppose n is 11, then we would check whether 2, 3, …, 5 (do we need to continue from 6?) are factors of 11 or not.
- Said differently, given n, the possible candidates of non-trivial factors are in [2, n/2].

candidates for non-trivial smaller factors of n: III

- In fact, we can do better. Non-trivial smaller factors are in $[2, \sqrt{n}]$. Why?
  - Factors appear in a pair. Once we know one, we can derive the other. For example, suppose n is 16, if 2 is a factor of 16, then 16/2 is also a factor of 16.
  - Possible ways to factor 16 are: 2 * 8, 4 * 4, 8 * 2. You can see 2 * 8 and 8 * 2 are redundant.

candidates for non-trivial smaller factors of n: II

- So we only need to look at the smaller factor among any factoring. Prove by contradiction that the biggest smaller factor of n cannot be larger than $\sqrt{n}$.
- Note that [2, $\sqrt{n}$] is a valid range only when n >= 4. Consider n = 1,2,3 separately.

Common mistake

- Print $\neq$ return
- If return type is not void, make sure return a value fit for the type in every branch.

no_digit function

- Define function no_digit, for a string, return true if none of its elements is a digit character, that is, one from ’0’ to ’9’, otherwise, return false.
  - One and done
  - ncounter one digit, return false IMMEDIATELY.
- For example, no_digit("abc") returns true and no_digit("ab1") returns false.
- Hint: you may use int isdigit(int ch) to test whether a character is digit letter ’0’ - ’9’ or not.

only_digit function

- Define function only_digit, for an array of chars, return true if all of its elements are digit characters, that is, one from ’0’ to ’9’, otherwise, return false.
  - One and done.
  - Encounter one non-digit, return false IMMEDIATELY.
- For example, given an array with elements ‘a’, ‘b’, ‘1’, only_digit returns …
- Given an array with elements ‘1’, ‘2’, ‘0’, only_digit returns ….
- Hint: you may use int isdigit(int ch) to test whether a character is digit letter ’0’ - ’9’ or not.